

Java Collections Framework

For a **3-year Java developer interview**, don't just memorize the definitions of `ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, and `HashMap`.

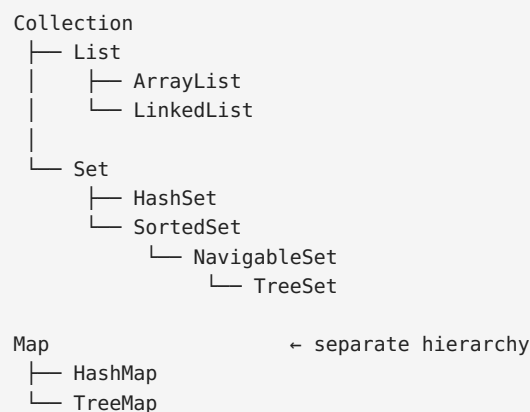
You should know:

- what problem each solves
 - internal data structure
 - ordering
 - duplicates
 - `null` behavior
 - time complexity
 - when to choose one over another
 - common interview traps
-

1. What is the Collections Framework?

The **Java Collections Framework (JCF)** provides interfaces and implementations for storing and manipulating groups of objects.

Important interfaces:



Important point:

`Map` is **not** a subtype of `Collection`.

2. ArrayList

What is it?

`ArrayList` is a **resizable array-based implementation of `List`**.

```
List<String> payments = new ArrayList<>();

payments.add("P101");
```

```
payments.add("P102");
payments.add("P103");
```

Characteristics

Property	ArrayList
Order	Maintains insertion order
Duplicates	Allowed
null	Allowed
Index access	Fast
Search by value	Usually O(n)
Insert at end	Amortized O(1)
Insert/remove middle	O(n)

Why is `get()` fast?

Because an `ArrayList` internally uses an array-like structure.

Conceptually:

index				
0	1	2	3	
P1	P2	P3	P4	

To get index 2:

```
payments.get(2);
```

Java can directly calculate where that element is.

Therefore:

```
get(index) → O(1)
```

3. Why is insertion in the middle O(n)?

Suppose:

```
[P1, P2, P3, P4]
```

Insert `PX` at index 1:

```
[P1, PX, P2, P3, P4]
```

Existing elements have to shift.

```
P2 → right
```

```
P3 → right  
P4 → right
```

Therefore:

```
add(end)      → amortized O(1)  
add(middle)   → O(n)  
remove(middle) → O(n)
```

4. LinkedList

LinkedList is a **doubly linked list** and implements both List and Deque.

Conceptually:

```
P1  
↓  
P2  
↓  
P3  
↓  
P4
```

Each node stores links to neighboring nodes.

```
prev | data | next
```

Characteristics

Property	LinkedList
Order	Maintains insertion order
Duplicates	Allowed
Random access	Slow
Add/remove at known node	O(1)
Search	O(n)
null	Allowed

5. Common misconception about LinkedList

People often say:

“LinkedList is faster for insertion.”

That’s incomplete.

Consider:

```
list.add(50, value);
```

Java first needs to **find position 50**, which can take O(n). Only after reaching the location is the

link change $O(1)$.

So:

LinkedList is beneficial when you already have the relevant node/iterator position and frequently insert/remove around it.

For typical application code, `ArrayList` is often the better default.

6. ArrayList vs LinkedList

Operation	ArrayList	LinkedList
<code>get(index)</code>	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$
Add at end	$O(1)$ amortized	$O(1)$
Add/remove middle by index	$O(n)$	$O(n)$
Memory overhead	Lower	Higher

Interview question

Which one would you normally choose?

Usually:

`ArrayList`, unless the workload specifically benefits from linked-list/deque behavior.

7. HashSet

`HashSet` stores **unique elements**.

```
Set<String> transactionIds = new HashSet<>();

transactionIds.add("TX100");
transactionIds.add("TX101");
transactionIds.add("TX100");
```

Result conceptually:

```
TX100
TX101
```

The duplicate `TX100` isn't added.

Characteristics

Property	HashSet
Duplicates	x
Ordering	No guaranteed encounter order
Search	Average $O(1)$

Insert	Average O(1)
Remove	Average O(1)
null	One null allowed

8. How does HashSet know something is duplicate?

This is an **important 3-year interview question**.

For objects, hashing and equality matter.

Conceptually:

```
hashCode()  
  ↓  
bucket  
  ↓  
equals()
```

Example:

```
Set<String> ids = new HashSet<>();  
  
ids.add("ABC");  
ids.add("ABC");
```

The set checks the hash/bucket and equality, so the second "ABC" isn't considered a new element.

Very important

If you create custom objects and use them in a `HashSet`, you should implement `equals()` and `hashCode()` consistently.

```
class Payment {  
  
    private String transactionId;  
  
    @Override  
    public boolean equals(Object o) {  
        ...  
    }  
  
    @Override  
    public int hashCode() {  
        ...  
    }  
}
```

General contract:

If two objects are equal according to `equals()`, they must have the same `hashCode()`.

9. TreeSet

`TreeSet` stores **unique elements in sorted order**.

```
Set<Integer> amounts = new TreeSet<>();

amounts.add(500);
amounts.add(100);
amounts.add(300);
```

Result:

```
100
300
500
```

Characteristics

Property	TreeSet
Duplicates	✗
Ordering	Sorted
Search	$O(\log n)$
Insert	$O(\log n)$
Remove	$O(\log n)$
null	Generally avoid; natural ordering doesn't support null

`TreeSet` is typically backed by a **balanced tree structure** (a Red-Black tree through `TreeMap`).

10. HashSet vs TreeSet

This is very common.

	HashSet	TreeSet
Duplicate	No	No
Ordering	No guaranteed order	Sorted
Average search	$O(1)$	$O(\log n)$
Implementation	Hash table	Balanced tree
Use when	Need uniqueness	Need uniqueness + sorted order

Example:

HashSet

```
Set<String> statuses = new HashSet<>();
```

Use when:

I only care about uniqueness.

TreeSet

```
Set<Integer> amounts = new TreeSet<>();
```

Use when:

I need uniqueness and sorted results.

11. HashMap

HashMap stores **key-value pairs**.

```
Map<String, Payment> payments = new HashMap<>();

payments.put("TX1001", payment1);
payments.put("TX1002", payment2);
```

Conceptually:

Key	Value
TX1001	→ Payment1
TX1002	→ Payment2

Characteristics

Property	HashMap
Key duplicates	✗
Value duplicates	✓
Ordering	No guaranteed encounter order
get()	Average O(1)
put()	Average O(1)
remove()	Average O(1)
null key	One
null values	Multiple

12. What happens when you put the same key?

```
Map<String, String> map = new HashMap<>();

map.put("TX100", "SUCCESS");
map.put("TX100", "FAILED");
```

There is still only one key:

TX100 → FAILED

The second `put()` replaces the old value.

13. HashMap Internal Working

This is **high priority for a 3-year Java interview**.

When you do:

```
map.put("TX100", payment);
```

conceptually:

```
Key
↓
hashCode()
↓
hash calculation
↓
bucket index
↓
store key/value
```

When you call:

```
map.get("TX100");
```

roughly:

```
Key
↓
hashCode()
↓
same bucket
↓
compare keys
↓
return value
```

That's why `HashMap` is typically $O(1)$ average for `get()` / `put()`.

14. Hash Collision

Different keys can produce the same bucket index.

```
Key A ┌──┐
      │  │ Bucket 5
Key B └──┘
```

This is called a **hash collision**.

Modern Java's `HashMap` can use a tree structure within a heavily-collided bucket after certain thresholds are reached, improving worst-case behavior in appropriate cases.

You don't need to memorize implementation thresholds unless specifically asked about JDK

internals, because they are implementation details and can change.

15. HashMap vs HashSet

This is another common question.

`HashSet` is essentially implemented using a `HashMap` internally.

Conceptually:

```
HashSet
  ↓
HashMap
  ↓
element stored as key
```

The set cares about the **element**.

The map cares about:

```
key → value
```

16. All Five Together

Suppose BPS needs these:

Ordered payment records

```
List<Payment> payments = new ArrayList<>();
```

Use:

| `ArrayList`

Frequent insertion/removal around iterator positions

```
List<Payment> payments = new LinkedList<>();
```

Use:

| `LinkedList` when the workload actually benefits from linked/deque operations.

Unique transaction IDs

```
Set<String> transactionIds = new HashSet<>();
```

Use:

| `HashSet`

Unique + sorted transaction IDs

```
Set<String> transactionIds = new TreeSet<>();
```

Use:

TreeSet

Transaction ID → Payment

```
Map<String, Payment> paymentMap = new HashMap<>();
```

Use:

HashMap

17. Complexity Cheat Sheet

Memorize this table.

Operation	ArrayList	LinkedList	HashSet	TreeSet	HashMap
Access by index	O(1)	O(n)	—	—	—
Search	O(n)	O(n)	O(1)*	O(log n)	O(1)*
Add	O(1)*	O(1)**	O(1)*	O(log n)	O(1)*
Remove	O(n)	O(1)**	O(1)*	O(log n)	O(1)*
Sorted	No	No	No	Yes	No

* Average/amortized complexity. ** When insertion/removal is performed at a known node/iterator position; finding that position can be O(n).

18. Collection vs Collections

Another interview trap.

Collection

An interface:

```
Collection<String> names;
```

Collections

A utility class:

```
Collections.sort(list);  
Collections.reverse(list);
```

Don't confuse them.

19. List vs Set vs Map

List

Ordered collection
Duplicates allowed
Access by index

Example:

```
List<Payment> payments;
```

Set

Unique elements

Example:

```
Set<String> transactionIds;
```

Map

Key → Value
Unique keys

Example:

```
Map<String, Payment> payments;
```

20. What happens with custom objects?

Suppose:

```
class Payment {  
    String transactionId;  
}
```

Then:

```
Set<Payment> payments = new HashSet<>();
```

Will it automatically understand that these are duplicates?

```
Payment("TX100")  
Payment("TX100")
```

Not based solely on the `transactionId` field.

You need correct `equals()` and `hashCode()` semantics.

This is an important connection:

```
HashMap / HashSet
    ↓
hashCode()
    ↓
equals()
```

Prepare this well.

21. Fail-Fast and Concurrent Modification

Another good 3-year topic:

```
for (String id : ids) {
    ids.remove(id);
}
```

This can result in:

```
ConcurrentModificationException
```

For safe removal during iteration:

```
Iterator<String> iterator = ids.iterator();

while (iterator.hasNext()) {
    String id = iterator.next();

    if (condition) {
        iterator.remove();
    }
}
```

There are also concurrent collections such as:

```
ConcurrentHashMap
CopyOnWriteArrayList
```

which are separate topics but worth knowing.

22. Common Interview Questions

Prepare these especially:

Basic

1. What is Java Collections Framework?
2. Difference between `List`, `Set`, and `Map`.
3. `ArrayList` vs `LinkedList`.
4. `HashSet` vs `TreeSet`.
5. `HashMap` vs `HashSet`.
6. Can `HashMap` have duplicate keys?

7. Can HashMap have null?
8. Can HashSet have null?
9. Does HashSet maintain insertion order?
10. Does HashMap maintain insertion order?

Intermediate

11. How does HashMap work internally?
12. How does HashSet identify duplicates?
13. Why do we override equals() and hashCode() together?
14. What is hash collision?
15. Why is HashMap average O(1)?
16. Why is TreeSet O(log n)?
17. Why is ArrayList get() O(1)?
18. Why is LinkedList random access O(n)?
19. When would you choose LinkedList over ArrayList?
20. What happens when HashMap grows?

Advanced 3-year questions

21. What happens if equals() is overridden but hashCode() isn't?
22. What happens if a key in HashMap is mutable?
23. How does HashMap handle collisions?
24. What's the difference between HashMap and ConcurrentHashMap?
25. What is fail-fast iteration?
26. What is ConcurrentModificationException?
27. Comparable vs Comparator?
28. How does TreeSet decide element ordering?
29. Why can TreeSet behave unexpectedly if comparison is inconsistent with equals()?
30. What collection would you choose for a high-read/high-write payment cache and why?

23. Project-based questions for your BPS interview

This is where you should connect Collections to your real work.

An interviewer might ask:

■ **“Where did you use HashMap in your project?”**

You could have something like:

```
Map<String, PaymentResponse> paymentResponseMap;
```

Then explain **why**:

■ “I needed to retrieve payment information using transaction ID, so a map provided efficient average constant-time lookup by key.”

Or:

■ **“Why did you use ArrayList?”**

■ “The operation primarily involved sequentially reading payment records, and we needed

index-based access in some cases, so ArrayList was appropriate.”

Don’t claim these examples unless they reflect your actual project. Use your real use case when you prepare.

24. One very important interview answer

Q: “Which collection would you choose?”

Don’t answer based only on memorization.

Use this decision process:

```
Need key → value?
  ↓
  Map
  ↓
Need sorted keys?
├ Yes → TreeMap
└ No  → HashMap

Need unique elements?
  ↓
  Set
  ↓
Need sorted?
├ Yes → TreeSet
└ No  → HashSet

Need duplicates/order?
  ↓
  List
  ↓
Usually → ArrayList
Special linked/deque workload → LinkedList
```

Final interview-level summary

```
ArrayList
→ Resizable array
→ Ordered
→ Duplicates allowed
→ Fast index access
→ Usually the default List

LinkedList
→ Doubly linked list
→ Ordered
→ Duplicates allowed
→ Slow random access
→ Useful for specific insertion/removal/deque workloads

HashSet
→ Unique elements
→ Hash-based
→ No guaranteed ordering
→ Average O(1) operations

TreeSet
```

- Unique + sorted
- Tree-based
- $O(\log n)$

HashMap

- Key → Value
- Unique keys
- Hash-based
- Average $O(1)$ get/put
- One null key, multiple null values

For your 3-year interview, prioritize these four concepts

1. HashMap internal working **2. equals() + hashCode()** **3. ArrayList vs LinkedList** **4. HashSet vs TreeSet**

Those are much more likely to lead to follow-up questions than simply asking you to define the five classes.